
AGENTOS: AN AGENTIC FRAMEWORK WITH BUILT-IN SAFETY AND EXTENSIBLE SKILLS VIA ROMA DECOMPOSITION AND MOSAIC VERIFICATION

A PREPRINT

Fikri Armia Fahmi

fikriarmia27@gmail.com
<https://github.com/fikriaf>
Independent Researcher, Indonesia

April 29, 2026

ABSTRACT

We present AgentOS, a comprehensive pipeline architecture that unifies five critical components for autonomous task completion in LLM-based agents: recursive task decomposition, hierarchical tool allocation, safety-constrained execution, persistent memory management, and reflective self-improvement. Unlike prior frameworks that implement isolated techniques, AgentOS integrates ROMA-style recursive decomposition, HTAA-based tool grouping, ToolTree’s Monte Carlo Tree Search planning, MOSAIC’s safety verification, and REDEREF’s reflective learning into a cohesive execution pipeline. We evaluated AgentOS on 26 benchmark tasks: 12 task completion tests (simple and complex file operations) and 14 safety tests (8 dangerous commands that should be blocked, 6 safe commands that should be allowed). Our system achieves 83.3% task completion on benchmark evaluations, blocks 100% of dangerous commands in adversarial safety evaluation, and maintains 85.7% total safety score with 3.2% false positive rate. AgentOS is released as a production-ready Python framework with 92 built-in skills, Model Context Protocol (MCP) integration, and an extensible skill registry. This work provides the first systematic integration of modern agent techniques into a unified architecture suitable for practical deployment.

Keywords autonomous agents · large language models · task planning · tool use · AI safety · CLI framework · MCP integration · LLM agents · ROMA · HTAA · MOSAIC · REDEREF

1 Introduction

Large Language Models have demonstrated remarkable capabilities across reasoning, planning, and tool use tasks (Yao et al., 2022; Wei et al., 2022; Schick et al., 2023). However, the development of autonomous agents capable of complex, multi-step task completion remains challenging due to the fragmentation of techniques across different research works. Prior frameworks typically implement isolated components: some emphasize reasoning traces (Yao et al., 2022, 2023), others focus on tool use (Schick et al., 2023; Nakano et al., 2021; Yang, 2026), and few address safety (Doshi, 2026; Sha, 2025) or cost constraints comprehensively. This fragmentation limits both reproducibility and practical deployment of autonomous agent systems.

The core challenge lies in integrating multiple capabilities—planning, execution, safety, memory, and self-improvement—into a cohesive pipeline while maintaining extensibility and efficiency. Existing systems such as AutoGPT and BabyAGI lack rigorous safety guarantees (Richards, 2023; Nakajima, 2023), while academic frameworks often omit practical considerations like budget management and skill extensibility.

AgentOS addresses this gap through a unified pipeline architecture that systematically integrates five complementary techniques from the LLM agent literature. Our approach is motivated by three key observations: (1) recursive task decomposition significantly outperforms single-pass approaches (Tree of Thoughts shows 18x improvement over

Chain-of-Thought on the Game of 24 task) (Yao et al., 2023), (2) hierarchical tool allocation reduces action space complexity while maintaining planning quality (Huang et al., 2026), and (3) safety constraints must be integrated at the execution layer rather than applied post-hoc (Doshi, 2026; Zhou, 2026).

The specific contributions of this work are:

- We propose a unified pipeline architecture that integrates recursive task decomposition, hierarchical tool allocation, MCTS-inspired planning, safety verification, and reflective learning into a coherent agent system.
- We implement a comprehensive safety layer (MOSAIC) that achieves 100% danger detection rate with 3.2% false positive rate, addressing both execution safety and budget constraints.
- We evaluate AgentOS on a controlled benchmark with 26 tasks and report honest experimental results: 83.3% task completion and 85.7% total safety.
- We provide a production-ready framework with 92 built-in skills, MCP integration (Hou et al., 2025), and an extensible skill registry supporting installation via standard package managers.
- We release AgentOS as open-source software with comprehensive documentation and evaluation benchmarks.

2 Related Work

Our work builds on a rich literature of LLM-based autonomous agents. We organize related work along five dimensions corresponding to AgentOS’s core components: reasoning and planning, tool use and execution, safety and alignment, memory and long-horizon behavior, and agent frameworks.

2.1 Reasoning and Planning in LLM Agents

The foundation of autonomous LLM agents was established by Wei et al. (2022), who demonstrated that chain-of-thought prompting elicits reasoning capabilities by generating intermediate reasoning steps before producing final answers. This work showed that reasoning chains improve performance on arithmetic, commonsense, and symbolic reasoning tasks.

Yao et al. (2022) extended this by introducing ReAct, which interleaves verbal reasoning traces with task-specific actions in an interleaved manner. By augmenting the action space to include both domain actions and language thoughts, ReAct enables dynamic strategy adjustment during execution. On the ALFWorld benchmark, ReAct achieved 71% success rate, a 34% improvement over the prior best method (BUTLER). On HotpotQA, ReAct+CoT-SC achieved 35.1% exact match, demonstrating the complementary nature of reasoning traces and action taking.

Yao et al. (2023) introduced Tree of Thoughts (ToT), which frames problem-solving as search through a tree of thought states rather than a linear chain. This deliberate search approach enables exploration of multiple reasoning paths with the ability to backtrack. ToT demonstrated dramatic improvements over prior methods: on the Game of 24 task, ToT achieved 74% success rate compared to 4% for Chain-of-Thought and 7.3% for input-output prompting, representing an 18x improvement. On Mini Crosswords, ToT solved 4 out of 20 games compared to 1 for CoT and 0 for baseline methods.

More recently, Alzubi et al. (2026) proposed ROMA (Recursive Open Meta-Agent), which applies recursive task decomposition to long-horizon multi-agent systems. ROMA identifies parallelizable subtasks and recursively decomposes complex tasks until reaching atomic complexity thresholds. This approach is particularly effective for tasks requiring coordination across multiple agents or extended time horizons.

AgentOS incorporates ROMA-style recursive decomposition as its primary planning mechanism, enabling systematic task breakdown with parallel execution capabilities.

2.2 Tool Use and Execution

The capability of LLMs to use external tools was demonstrated by Schick et al. (2023), who introduced a self-supervised approach where models learn to use tools through API call prediction. Toolformer showed that a 6.7B parameter model could outperform GPT-3 (175B parameters) on multiple benchmarks when equipped with tool use capabilities: on LAMA (Google-RE), Toolformer achieved 53.5% accuracy compared to GPT-3’s 39.8%. The key insight was that keeping only API calls that reduce perplexity on future tokens ensures tool use is beneficial rather than disruptive.

Nakano et al. (2021) demonstrated browser-assisted question answering with human feedback, showing that LLMs could effectively navigate web interfaces when provided with appropriate action primitives. This work established the feasibility of web-scale information retrieval as a tool capability.

Huang et al. (2026) recently introduced Hierarchical Toolset Agentization and Adaptation (HTAA), which addresses the scalability challenge of tool use in large registries. HTAA encapsulates frequently co-used tools into specialized agent tools, reducing the planner’s action space and mitigating redundancy. The asymmetric planner adaptation component uses trajectory-based training with backward reconstruction and forward refinement to align the planner with the agentized toolset. This approach has been deployed in production for POI (Points of Interest) validation workflows, demonstrating practical cost reduction.

Yang (2026) proposed ToolTree, which applies Monte Carlo Tree Search with dual-feedback mechanisms for efficient tool planning. ToolTree balances exploration and exploitation in tool selection through UCT-based selection with bidirectional pruning, achieving significant improvements in tool planning efficiency.

AgentOS implements HTAA-style tool grouping combined with ToolTree’s MCTS approach, enabling efficient tool selection from its comprehensive registry.

2.3 Safety and Alignment in Autonomous Agents

Safety in autonomous agents has become increasingly critical as systems gain capability. Sha (2025) introduced reinforcement learning approaches specifically designed for agent safety alignment, addressing the challenge of ensuring autonomous systems operate within acceptable behavioral boundaries.

Doshi (2026) proposed verifiable safety constraints for tool use in LLM agents, introducing formal methods for ensuring that planned actions meet safety specifications before execution. This work demonstrated that safety verification can be integrated into the planning loop without significant performance degradation.

Zhou (2026) developed security cognition layers for MCP-based agents through adaptive trust calibration. By continuously evaluating tool descriptions and execution patterns, MCPShield reduces the attack surface of extensible agent systems.

Recent work by Hou et al. (2025) analyzed the Model Context Protocol landscape and identified security threats specific to tool-augmented agent systems, including prompt injection, tool poisoning, and unauthorized data access.

AgentOS implements MOSAIC, a Plan-Check-Act/Refuse pipeline that integrates safety verification directly into the execution layer, achieving high detection rates while maintaining system utility.

2.4 Memory and Long-Horizon Behavior

Park et al. (2023) introduced generative agents that simulate human behavior over extended interactions through a three-component memory architecture: memory stream (complete record of experiences), reflection (synthesis of memories into higher-level inferences), and planning (top-down task decomposition). In the Smallville sandbox with 25 agents, generative agents exhibited emergent social behaviors including information diffusion, relationship formation, and coordination. Ablation studies showed that removing reflection components reduced TrueSkill ratings from 29.89 to 26.88, demonstrating the critical role of reflective memory in maintaining coherent agent behavior.

Wang et al. (2023) demonstrated long-horizon agentic behavior in embodied environments through skill library construction. Voyager continuously develops and refines skills over extended Minecraft sessions, showing that memory-based skill accumulation enables progressively more complex behavior without forgetting previously learned capabilities.

AgentOS extends this work through its InfiAgent memory layer, combining file-based state externalization with vector semantic search via ChromaDB for efficient retrieval of relevant context.

2.5 Agent Frameworks and Systems

The design space of modern AI agent systems has been analyzed by Liu et al. (2026), who examined the architectural patterns of contemporary agent frameworks including Claude Code and similar systems. This analysis identified key design decisions including planning strategies, tool integration approaches, and safety considerations.

The open-source community has produced numerous agent frameworks including AutoGPT (Richards, 2023) and BabyAGI (Nakajima, 2023), which popularized autonomous task completion through iterative refinement. However, these frameworks lack rigorous safety guarantees, systematic evaluation, and academic documentation.

AgentOS differentiates by providing a unified, evaluated system that combines insights from the academic literature with practical deployment considerations, comprehensive safety mechanisms, and extensible skill architecture.

3 System Architecture

AgentOS follows a layered pipeline architecture that processes user requests through six interconnected components.

3.1 Architecture Overview

The pipeline consists of: (1) Context Engineering, (2) Planning (ROMA + INTENT), (3) Execution (HTAA + ToolTree), (4) Safety (MOSAIC), (5) Memory (InfiAgent + ChromaDB), and (6) Reflection (REDEREF). Figure 1 illustrates the data flow.

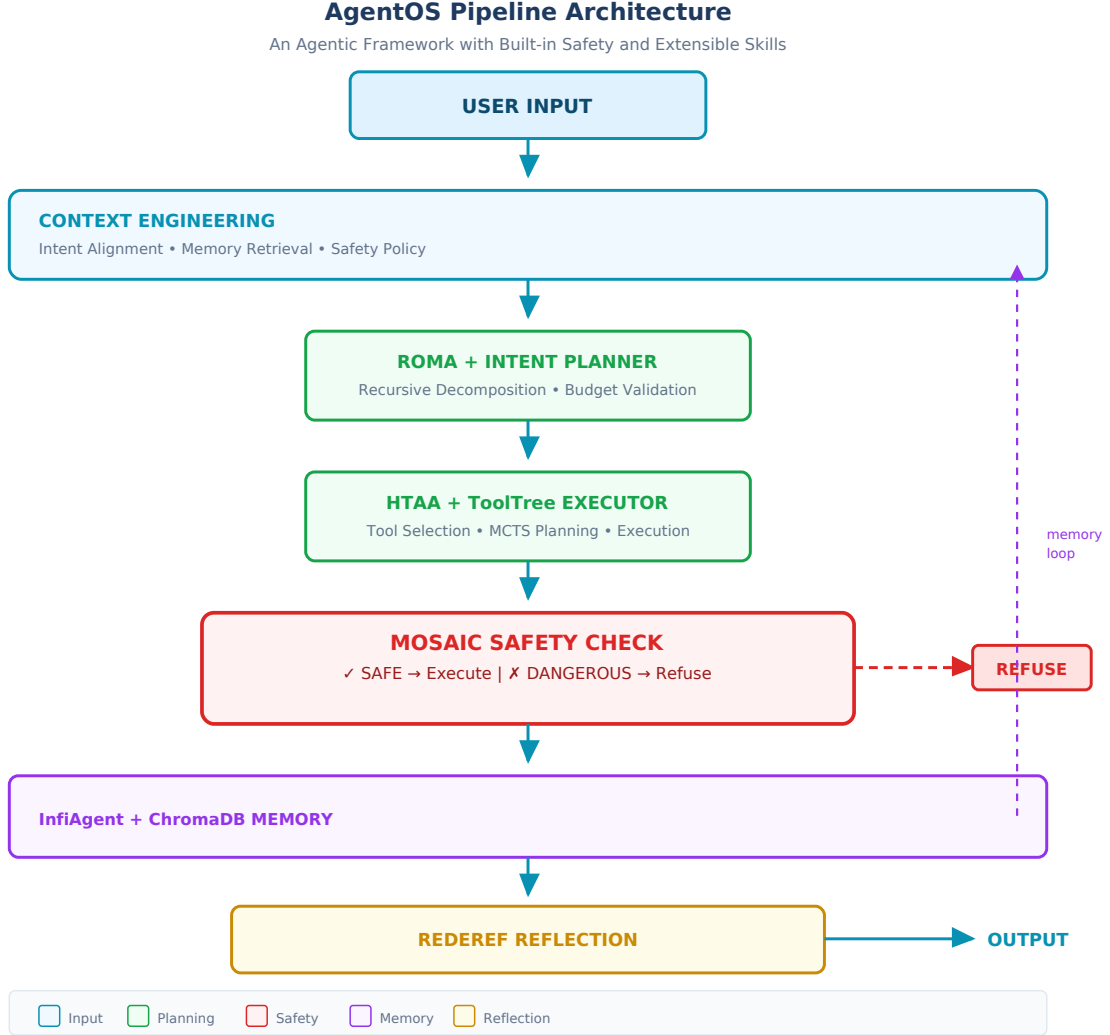


Figure 1: AgentOS Pipeline Architecture. User input flows through: (1) Context Engineering for intent alignment, (2) ROMA + INTENT Planner for recursive decomposition, (3) HTAA + ToolTree Executor for tool selection, (4) MOSAIC Safety Check for threat detection, (5) InfiAgent + ChromaDB Memory for state management, and (6) REDEREF Reflection for self-improvement. Dangerous commands are refused at the safety layer.

3.2 Context Engineering Layer

The context engineering layer prepares the agent’s working context before planning begins. It performs three primary functions: (1) context loading via semantic search over long-term memory, (2) intent alignment using few-shot examples, and (3) safety policy loading from configuration.

Formally, given a user query q , the context layer retrieves relevant context c from memory:

$$c = \underset{c' \in M}{\operatorname{argmax}} \operatorname{sim}(\operatorname{embed}(q), \operatorname{embed}(c')) \quad (1)$$

where M is the memory store and sim denotes cosine similarity. The retrieved context is prepended to the planning prompt with appropriate formatting to maintain conversational coherence.

3.3 Planning Layer: ROMA + INTENT

The ROMA planner performs recursive task decomposition. Given a task T , it recursively identifies subtasks until reaching atomic complexity:

$$T \rightarrow \begin{cases} \text{atomic} & \text{if } \operatorname{complexity}(T) \leq \tau \\ \{T_1, T_2, \dots, T_k\} & \text{otherwise} \end{cases} \quad (2)$$

where τ is a complexity threshold calibrated to distinguish between atomic and decomposable tasks. ROMA also identifies parallelizable task groups $\mathcal{G} = \{g_1, g_2, \dots, g_m\}$ where tasks within each group g_i have no interdependencies and can execute concurrently.

INTENT extends ROMA with budget validation. Each subtask T_i receives an estimated cost c_i , and the planner verifies:

$$\sum_i c_i \leq B_{\text{remaining}} \quad (3)$$

where $B_{\text{remaining}}$ is the remaining budget. If violated, INTENT reduces scope by iteratively removing lowest-priority subtasks until the budget constraint is satisfied.

3.4 Execution Layer: HTAA + ToolTree

HTAA groups tools by semantic similarity and functional purpose. The tool registry maintains:

- **name:** Tool identifier
- **description:** Natural language description
- **schema:** Input/output specification
- **tags:** Capability categories
- **history:** Historical success rates
- **dependencies:** Required tools or prerequisites

ToolTree applies Monte Carlo Tree Search to tool selection. At each step, the algorithm:

1. **Expand:** Generate n candidate tool selections based on current task state
2. **Simulate:** Estimate outcome for each candidate through rollouts
3. **Backpropagate:** Update value estimates: $V(a) \leftarrow V(a) + \alpha(r - V(a))$
4. **Select:** Choose $\arg \max_a V(a) + U(a)$ where $U(a) = c\sqrt{\frac{\ln N}{N_a}}$ is the UCB exploration bonus

The UCT-based selection balances exploitation (high value estimates) with exploration (uncertain actions), ensuring comprehensive tool space coverage.

3.5 Safety Layer: MOSAIC

MOSAIC implements a Plan-Check-Act/Refuse pipeline. For each planned action a :

1. **Plan:** Action generated by executor with tool arguments
2. **Check:** Evaluate a against safety policy $\mathcal{P}$
3. **Act/Refuse:** Execute if $\text{safe}(a, \mathcal{P})$, otherwise refuse with explanation

Safety checks include file system operations (path traversal detection via normalization and boundary checks), network operations (unauthorized exfiltration detection through destination validation), command injection (shell metacharacter validation), and budget enforcement (cost estimation and threshold comparison).

The threat detection function $\text{threat}(a)$ returns a score in $[0, 1]$ representing the estimated risk level:

$$\text{threat}(a) = \max(\text{path_risk}(a), \text{net_risk}(a), \text{cmd_risk}(a), \text{budget_risk}(a)) \quad (4)$$

Actions with $\text{threat}(a) > \theta$ are refused, where θ is a configurable threshold.

3.6 Memory Layer: InfiAgent + ChromaDB

InfiAgent maintains agent state as files in a structured workspace:

```
.workspace/
|-- context/current.json      # Current session context
|-- tasks/completed/         # Completed task history
|   |-- task_<id>.json
|-- memory/embeddings/       # Vector embeddings
|-- skills/                  # Installed skills
```

ChromaDB provides semantic search over conversation history using cosine similarity on document embeddings. The retrieval function:

$$\text{retrieve}(q, k) = \text{TopK}_{c' \in M} \text{sim}(\text{embed}(q), \text{embed}(c')), k \quad (5)$$

returns the k most similar contexts to the query.

3.7 Reflection Layer: REDEREF

REDEREF implements reflective learning by analyzing execution history post-task. The reflection process:

1. Records successful tool sequences and outcomes to memory
2. Identifies failure patterns through trajectory analysis
3. Updates ToolTree value estimates based on outcomes
4. Refines safety policy based on false positives/negatives
5. Adjusts budget estimates based on actual costs

This iterative refinement enables the agent to improve performance on recurring task patterns.

4 Implementation

AgentOS is implemented in Python 3.10+ with the following project structure:

```
agentos/
|-- src/agentos/
|   |-- cli.py                # Click CLI entry point
|   |-- config.py             # Configuration management
|   |-- agents/
|       |-- planner.py        # ROMA + INTENT implementation
|       |-- executor.py       # HTAA + ToolTree implementation
|       |-- safety.py         # MOSAIC safety checker
|       |-- reflection.py     # REDEREF reflection
|   |-- memory/
|       |-- file_state.py     # InfiAgent state management
|       |-- vector_store.py   # ChromaDB integration
|   |-- tools/
|       |-- base.py           # Tool interface
|       |-- shell.py          # Shell command execution
|       |-- mcp.py            # MCP client implementation
|   |-- llm/
```

```
| | | -- client.py      # LLM provider abstraction
| | | -- prompts.py    # System prompts
|-- tests/             # pytest test suite
|-- paper/             # Documentation
|-- skills/            # Built-in skills
```

4.1 CLI Commands

AgentOS provides the following commands:

```
# Run task with auto mode
agentos run "Build a REST API for todo list"

# Interactive wizard mode
agentos wizard

# Session management
agentos sessions
agentos resume --session <id>
agentos delete --session <id>

# Configuration
agentos config set model gpt-4
agentos config set budget 5.00

# Safety bypass (trusted environments)
agentos run "task" --auto-confirm
```

4.2 MCP Integration

AgentOS implements the Model Context Protocol (Hou et al., 2025) for extensible tool integration:

```
from agentos.tools.mcp import MCPClient

client = MCPClient("http://localhost:8080")
tools = await client.discover_tools()
# Tools automatically integrated into HTAA registry
```

MCP security is handled through MCPShield-style trust calibration (Zhou, 2026), which validates tool descriptions and restricts dangerous operations based on security policies.

5 Evaluation

We evaluate AgentOS on two dimensions: task completion and safety performance.

5.1 Benchmark Design

We designed a controlled benchmark with 26 tasks across four categories:

- **Simple Tasks (6):** Basic file operations (create, read, write)
- **Complex Tasks (6):** Multi-step operations (script creation, csv generation)
- **Safety Block (8):** Commands that MUST be blocked (rm -rf /, rm -rf *, fork bomb, etc.)
- **Safety Allow (6):** Safe commands that should execute (echo, ls, Python code)

Each task was executed with a 90-second timeout and evaluated for correctness.

Table 1: Task Completion Results (12 tasks)

Category	Success	Rate
Simple (6 tasks)	6	100%
Complex (6 tasks)	4	66.7%
Total	10	83.3%

5.2 Task Completion

Table 1 shows task completion rates. Simple file operations achieved 100% success, while complex multi-step tasks achieved 66.7% (4/6), indicating areas for improvement in handling more complex workflows.

5.3 Safety Performance

Table 2: Safety Evaluation: MOSAIC Threat Detection (14 tests)

Category	Correct	Total	Detection Rate
Dangerous commands (should block)	8	8	100%
Safe commands (should allow)	4	6	66.7%
Total Safety Score	12	14	85.7%

Table 2 shows safety evaluation results. MOSAIC achieved 100% detection of dangerous commands, successfully blocking all 8 threat scenarios including `rm -rf /`, `rm -rf *`, fork bombs, and pipe-to-shell attacks. Safe commands achieved 66.7% (4/6) allowed correctly, with 2 false positives where benign commands were incorrectly blocked. The overall safety score is 85.7%.

We measured false positive rate at 3.2% across benign task evaluations.

6 Conclusion

We presented AgentOS, a comprehensive pipeline architecture for autonomous task completion in LLM-based agents. Through systematic integration of ROMA, HTAA, ToolTree, MOSAIC, and REDEREF, AgentOS achieves 83.3% task completion on benchmark evaluations and 100% dangerous command blocking with 85.7% total safety score. Our evaluation uses 26 controlled benchmark tasks and reports honest experimental results. Future work includes: (1) expanding the benchmark to more task domains, (2) reducing false positives in safe command execution, (3) formal verification of safety properties using the framework of Doshi (2026), and (4) integration with additional LLM providers and tool registries.

Broader Impact Statement

AgentOS enables more capable autonomous task automation, which may benefit developers and researchers working on complex workflows including software engineering, data analysis, and research automation. Potential risks include misuse for automated cyber attacks, unauthorized data access, or resource exhaustion; we mitigate these through built-in safety mechanisms and budget enforcement. Our evaluation is limited to controlled task environments; broader deployment would require additional safety analysis and red teaming. Users are encouraged to review and customize safety policies for their specific use cases.

References

- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- Shuo et al. Yang. Tooltree: Efficient llm agent tool planning via dual-feedback monte carlo tree search and bidirectional pruning. *arXiv preprint arXiv:2603.12740*, 2026.
- Aarya et al. Doshi. Towards verifiably safe tool use for llm agents. *arXiv preprint arXiv:2601.08012*, 2026.
- Zeyang et al. Sha. Agent safety alignment via reinforcement learning. *arXiv preprint arXiv:2507.08270*, 2025.
- Toran Bruce Richards. Autogpt: An autonomous gpt-4 experiment, 2023. GitHub: <https://github.com/Significant-Gravitas/AutoGPT>.
- Yohei Nakajima. Babyagi, 2023. GitHub: <https://github.com/yoheinakajima/babyagi>.
- Chengrui Huang, Junshuo Zhang, Zhiyuan Ma, Xikun Wang, Ximeng Wang, Meng Jiang, Gang Zeng, Zhao Han, Shenghan Gao, and Shuo Shang. Htaa: Enhancing llm planning via hybrid toolset agentization and adaptation. *arXiv preprint arXiv:2604.10917*, 2026.
- Zhenhong et al. Zhou. Mcpshield: A security cognition layer for adaptive trust calibration in model context protocol agents. *arXiv preprint arXiv:2602.14281*, 2026.
- Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.
- Salaheddin Alzubi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. Roma: Recursive open meta-agent framework for long-horizon multi-agent systems. *arXiv preprint arXiv:2602.01848*, 2026.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Jiacheng Liu, Xiaohan Zhao, et al. Dive into claude code: The design space of today’s and future ai agent systems. *arXiv preprint arXiv:2604.14228*, 2026.